

DEVOPS MASTER COURSE

Volume 8A

Google Cloud Foundations

Chapters 1-10

**Cloud Fundamentals • Global Infrastructure • Resource Hierarchy • IAM • Service Accounts •
gcloud • Compute Engine • VPC • Billing • Capstone**

2026 Edition

About This Part

Volume 8A establishes the operating model required to use Google Cloud safely and professionally. It begins with cloud principles and Google Cloud global infrastructure, then moves through resource hierarchy, identity, command-line automation, compute, networking, cost governance, and an integrated foundation capstone.

The objective is not simply to create resources. A production engineer must understand ownership, inherited policy, billing boundaries, blast radius, identity paths, quotas, location constraints, logging, cleanup, and evidence. Every lab therefore includes validation and cost-control steps.

Volume 8 Structure

Part	Coverage	Status
8A	Foundations: hierarchy, IAM, tools, compute, VPC, billing	This book
8B	Core infrastructure: managed instance groups, load balancing, DNS, storage, hybrid connectivity, DR	Next
8C	Databases, analytics, data engineering, AI-ready data platforms, migration	Planned
8D	GKE, Cloud Run, functions, APIs, Pub/Sub, Eventarc, workflows	Planned
8E	Terraform, Cloud Build, observability, security, governance, FinOps	Planned
8F	Enterprise projects, incidents, certifications, interviews, final capstone	Planned

Safety, Security, and Cost Controls

- Use a dedicated learning project and never experiment in an employer or production organization without written authorization.
- Enable budgets and alerts before labs. Budgets notify; they are not a substitute for cleanup, quotas, and automation.
- Prefer short-lived credentials, Cloud Shell, service account impersonation, and Workload Identity Federation. Do not create service account keys unless a documented exception requires them.
- Use least privilege, groups instead of individual grants, and separate human identities from workload identities.
- Choose regions deliberately for latency, service availability, compliance, resilience, and cost. Remove public IP addresses unless they are required.
- Delete temporary VMs, disks, static IPs, snapshots, and projects after evidence is captured.

Table of Contents

Chapter 1 - Cloud Computing and Google Cloud

Chapter 2 - Google Cloud Global Infrastructure and Reliability

Chapter 3 - Organizations, Folders, Projects, and Billing Accounts

Chapter 4 - Console, Cloud Shell, gcloud CLI, APIs, and SDKs

Chapter 5 - Cloud Identity and IAM Fundamentals

Chapter 6 - Service Accounts, Workload Identity Federation, and Privileged Access

Chapter 7 - Resource Governance: Organization Policy, Tags, Labels, Quotas, and APIs

Chapter 8 - Compute Engine Foundations

Chapter 9 - VPC Networking Foundations

Chapter 10 - Cloud Billing, FinOps, and Foundation Capstone

Appendix A - Google Cloud Command Reference

Appendix B - Cross-Service Troubleshooting Matrix

Appendix C - Foundation Production-Readiness Checklist

Appendix D - Glossary and Official Documentation Index

Chapter 1 - Cloud Computing and Google Cloud

Understanding cloud operating models, shared responsibility, and the Google Cloud service landscape

Learning Objectives

- Explain cloud computing through business and engineering outcomes.
- Differentiate IaaS, PaaS, managed services, and serverless platforms.
- Describe shared responsibility and the control-versus-operations trade-off.
- Map common DevOps capabilities to Google Cloud services.

1.1 Why Cloud Changes Engineering

Cloud computing provides on-demand access to shared infrastructure and managed services through programmable APIs. The important change is not that servers are remote. The important change is that capacity, identity, networking, policy, and observability become software-defined and can be delivered through repeatable automation.

- Elastic capacity reduces procurement delay but requires scaling limits and cost governance.
- Managed services reduce undifferentiated operations but still require configuration, data protection, availability design, and monitoring.
- Global infrastructure enables regional architecture but does not make every workload automatically resilient.
- APIs improve repeatability while increasing the importance of identity, audit, and change control.

1.2 Service Models and Responsibility

Service models represent different allocations of control and operational burden. Compute Engine gives operating-system control. Google Kubernetes Engine provides managed Kubernetes control planes with workload and cluster responsibilities shared by the customer. Cloud Run removes server and cluster management for suitable containerized applications. Managed databases transfer patching and replication work while leaving schema, queries, access, and recovery objectives with the customer.

Model	Google Cloud Examples	Customer Focus
Infrastructure	Compute Engine, Persistent Disk, VPC	OS, runtime, application, data, scaling, backup, access.
Managed platform	GKE, Cloud SQL, Memorystore	Workload configuration, data model, security, availability, cost.
Serverless application	Cloud Run, Cloud Run functions	Code/container, identity, concurrency, limits, observability.
Managed analytics	BigQuery, Dataflow, Pub/Sub	Data quality, schemas, permissions, pipelines, governance.

1.3 Google Cloud for DevOps

Google Cloud combines compute, networking, data, security, operations, and developer services behind a consistent project and IAM model. DevOps engineers commonly use Cloud Build or external CI systems, Artifact Registry, Compute Engine, GKE, Cloud Run, Cloud Storage, Cloud SQL, Cloud Monitoring, Cloud Logging, Secret Manager, and Infrastructure as Code.

- Use projects as strong administrative, quota, and billing boundaries.

- Use labels and tags for ownership, policy, automation, and cost allocation.
- Use Cloud Audit Logs and centralized observability to preserve accountability.
- Prefer managed identities and short-lived credentials over static secrets.

1.4 Architecture Selection

Choose the smallest operational surface that satisfies the requirements. A simple stateless API might fit Cloud Run; a legacy operating-system workload might require Compute Engine; a Kubernetes platform can be justified when scheduling, ecosystem, or portability requirements outweigh its complexity. Selection must account for reliability, security, team capability, latency, compliance, lock-in, and total cost.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Unexpected cost after a lab	Resources continue running or attached storage/IPs remain	Review Billing reports and asset inventory; stop or delete compute, disks, addresses, and snapshots.
Managed service still fails	Shared responsibility misunderstood	Inspect configuration, quotas, IAM, dependencies, application errors, and recovery design.
Team chooses Kubernetes for every app	Platform preference instead of requirements	Use an architecture decision record comparing Cloud Run, GKE, Compute Engine, and managed services.

Hands-on Lab 1: Create a Google Cloud Learning Environment

Prerequisites

- A personal Google account or approved Cloud Identity account.
- A payment method or approved training billing account.

Implementation

1. Create or select a dedicated Google Cloud project with a globally unique project ID.
2. Link the project to a billing account and record the project number and billing account boundary.
3. Open Cloud Shell and run `gcloud config list`.
4. Enable only the APIs needed for later labs.
5. Create a budget and at least three notification thresholds.
6. Record an owner, environment, expiry date, and cleanup plan in the project documentation.

Validation Evidence

- The active account and project are correct.
- A budget is visible for the learning scope.
- No production organization or shared project is used.
- The project has an explicit owner and expiry decision.

Cleanup

- Disable unused APIs if required.
- Do not delete the project yet; it is reused in later labs.

Interview Questions and Model Answers

1. What is the main engineering benefit of cloud?

Programmable, on-demand access to infrastructure and managed capabilities with measurable consumption.

2. Does a managed service remove customer responsibility?

No. It changes responsibility; customers still own configuration, identities, data, application behavior, and recovery objectives.

3. How do you choose between Compute Engine and Cloud Run?

Compare OS/control requirements, workload shape, scaling, networking, runtime constraints, team skill, and operations cost.

4. Why are projects important?

They are core boundaries for resources, APIs, IAM, quotas, billing association, and lifecycle management.

Chapter Summary

- Cloud is an operating model based on programmable services.
- Service selection is a control-versus-operations decision.
- Shared responsibility remains essential.
- Projects, identity, audit, automation, and cost controls form the foundation.

Chapter 2 - Google Cloud Global Infrastructure and Reliability

Designing location-aware systems using regions, zones, edge infrastructure, and failure domains

Learning Objectives

- Differentiate regions, zones, and global resources.
- Select locations using availability, latency, compliance, and cost requirements.
- Explain zonal, regional, and multi-region reliability patterns.
- Identify hidden regional and global dependencies.

2.1 Regions and Zones

A Google Cloud region is an independent geographic area that contains multiple zones. A zone is a deployment area within a region and represents a failure domain for zonal resources. Many Compute Engine resources are zonal or regional, while other resources such as VPC networks are global.

Scope	Examples	Design Implication
Zonal	VM instances, zonal managed instance groups	A zone failure can affect the resource; distribute replicas.
Regional	Subnets, regional managed instance groups, regional disks	Use multiple zones and understand regional failure scenarios.
Global	VPC networks, many firewall policies, global load-balancing resources	Global scope simplifies connectivity but can enlarge policy blast radius.
Multi-region/service-defined	Some storage and data services	Behavior depends on the product replication and consistency model.

2.2 Location Selection

A location decision must be explicit. Evaluate user latency, data residency, service and machine-type availability, quotas, carbon and sustainability requirements, inter-region data transfer, disaster-recovery strategy, and operational support. The nearest region is not always the correct region if a required service, compliance control, or recovery pairing is unavailable.

2.3 Reliability Patterns

A single VM in one zone is a single failure domain. Zonal resilience uses multiple instances in one zone but does not protect against zone loss. Regional patterns distribute replicas across zones, commonly through regional managed instance groups and regional services. Multi-region patterns add geographic resilience but require stronger data, routing, deployment, and operations design.

- Define service-level objectives and business impact before selecting redundancy.
- Treat capacity, quota, identity, DNS, secrets, observability, and dependencies as part of failover.
- Test restoration and failover; configuration alone is not evidence.
- Document failback and reconciliation, not only failover.

2.4 Failure Domains and Blast Radius

A resource can be regional while still depending on a global control plane or a third-party service. A globally scoped VPC or IAM policy can affect workloads in many regions. Use projects, folders, service perimeters,

regional cells, and staged rollout to bound failures. Reliability is the ability to continue correct service, not merely keep processes running.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
VM unavailable after zone event	Single-zonal deployment	Use replicas across zones, a regional managed instance group, health checks, and load balancing.
Service cannot deploy in selected region	Product, machine type, or quota unavailable	Check location documentation and quotas before architecture approval.
Failover region cannot handle traffic	Insufficient quota or warm capacity	Reserve/test capacity, define load shedding, and include quota in game days.

Hands-on Lab 2: Build a Location Decision Record

Prerequisites

- The learning project and access to Google Cloud location documentation.

Implementation

- Choose a sample customer-facing application and identify its users and data classes.
- Shortlist two candidate regions and compare latency, service availability, quotas, compliance, and cost.
- Classify each proposed resource as zonal, regional, global, or service-defined.
- Define the primary failure domains and recovery objectives.
- Create an architecture decision record with rejected alternatives.
- Write a test plan for zone failure and regional recovery.

Validation Evidence

- The selected region supports required services.
- The design contains no accidental single-zone critical dependency.
- RTO and RPO are measurable.
- Rejected options and assumptions are documented.

Cleanup

- No cloud resources are required; retain the decision record as portfolio evidence.

Interview Questions and Model Answers

1. What is a zone?

A deployment area within a region that functions as a failure domain for zonal resources.

2. Is a VPC network regional?

No. A Google Cloud VPC network is global, while its subnets are regional.

3. When is multi-region justified?

When business continuity, latency, regulatory, or scale requirements justify the added data and operational complexity.

4. Why must quotas be included in disaster recovery?

A recovery plan can fail if the target location cannot allocate the required capacity.

Chapter Summary

- Regions contain zones and define geographic placement.
- Resource scope changes failure behavior.
- Reliability must include dependencies, quota, data, and operations.
- Location and recovery decisions require documented evidence.

Chapter 3 - Organizations, Folders, Projects, and Billing Accounts

Building a scalable Google Cloud resource hierarchy with explicit ownership and lifecycle boundaries

Learning Objectives

- Explain the Google Cloud resource hierarchy.
- Differentiate organization, folder, project, resource, and billing boundaries.
- Design project and folder structures for teams and environments.
- Apply inherited policy and lifecycle governance safely.

3.1 Resource Hierarchy

The Google Cloud hierarchy normally begins with an organization resource, followed by optional folders, projects, and service-specific resources. Every resource below the root has one parent. IAM allow policies and organization policies can be inherited down the hierarchy, which makes hierarchy design a security and governance decision rather than a cosmetic folder exercise.

Node	Purpose	Typical Controls
Organization	Enterprise root and central governance	Organization policies, domain restrictions, audit and security administration.
Folder	Group projects by business unit, environment, platform, or control boundary	Inherited IAM, policies, tags, visibility, delegated administration.
Project	Core resource, API, quota, lifecycle, and billing-association boundary	Project IAM, enabled APIs, labels, budgets, service resources.
Resource	VM, bucket, database, network component, and other service object	Service-specific IAM, configuration, encryption, logging.

3.2 Projects as Operating Units

Projects have a human-readable name, a globally unique project ID, and an immutable project number. The project ID is often used in commands and endpoints; the project number is used by service agents and some identity relationships. Project deletion has broad impact and must be protected by process, permissions, and retention planning.

- Separate production from nonproduction.
- Separate shared networking, centralized logging, security, and workload projects when scale and controls justify it.
- Avoid one giant project that combines unrelated owners and lifecycles.
- Avoid excessive project fragmentation without automation and ownership.

3.3 Billing Accounts and Cost Boundaries

A Cloud Billing account pays for linked projects but is not the parent of those projects in the resource hierarchy. Billing permissions and resource permissions are separate. This separation allows finance teams to manage payment and reporting without receiving workload administration, and platform teams to manage resources without payment-method authority.

3.4 Enterprise Hierarchy Pattern

A common enterprise pattern uses folders for platform, security, shared services, production, and nonproduction branches. Projects then represent workloads, environments, or shared capabilities. Groups receive roles at the narrowest sustainable level, and inherited controls are tested before broad rollout.

```

Organization
├── Platform
│   ├── Network Host Project
│   └── CI/CD Project
├── Security
│   ├── Logging Project
│   └── Security Operations Project
├── Production
│   ├── Payments Production Project
│   └── Customer Portal Production Project
└── Nonproduction
    ├── Development Projects
    └── Test Projects
  
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
User can access many projects unexpectedly	Role inherited from organization or folder	Use policy troubleshooter and inspect inherited bindings; remove or narrow the upstream grant.
Project cannot be linked to billing	Missing billing-account and project permissions	Verify Billing Account User plus project billing-management permissions and organization restrictions.
Policy change breaks child projects	Broad inherited policy without staged testing	Test in a lower folder, inspect effective policy, use change review and rollback.

Hands-on Lab 3: Design a Google Cloud Resource Hierarchy

Prerequisites

- A synthetic company scenario; organization access is optional.

Implementation

1. Identify business units, environments, platform services, security functions, and ownership boundaries.
2. Design an organization and folder hierarchy with no more levels than needed.
3. Define project naming, IDs, labels, owners, and lifecycle states.
4. Separate resource IAM from billing IAM responsibilities.
5. Create a permission matrix for organization, folder, and project administrators.
6. Document a project creation, suspension, and deletion workflow.

Validation Evidence

- Every project has one accountable owner.
- Production and nonproduction are separated.
- Central services have explicit consumers and delegated administration.
- Billing and resource permissions are separated.

Cleanup

- No live organization changes are required; retain the hierarchy diagram and permission matrix.

Interview Questions and Model Answers

1. What is the fundamental operating unit in Google Cloud?

The project, which contains service resources and provides API, quota, lifecycle, and billing-association boundaries.

2. Does a billing account own projects in the hierarchy?

No. It pays for linked projects but is separate from the Resource Manager parent hierarchy.

3. Why use folders?

To group projects and apply inherited IAM and organization policies at scalable control boundaries.

4. What is the risk of granting roles at the organization level?

The grant can inherit to a very large resource set and create excessive privilege or blast radius.

Chapter Summary

- Organizations, folders, projects, and resources form the governance hierarchy.
- Projects are core operating and lifecycle boundaries.
- Billing and resource administration are separate.
- Inheritance requires careful scope, testing, and ownership.

Chapter 4 - Console, Cloud Shell, gcloud CLI, APIs, and SDKs

Operating Google Cloud consistently through graphical, command-line, and programmatic interfaces

Learning Objectives

- Use the Google Cloud console and Cloud Shell safely.
- Configure and validate gcloud accounts, projects, regions, and zones.
- Understand APIs, client libraries, REST, and Application Default Credentials.
- Build repeatable administrative workflows with evidence.

4.1 Management Interfaces

The Google Cloud console is useful for discovery, guided configuration, and inspection. Cloud Shell provides a browser-based environment with the Google Cloud CLI and common tools preinstalled. The gcloud CLI supports repeatable administration. Client libraries and REST APIs enable applications and automation to call services programmatically.

Interface	Best Use	Main Risk
Console	Learning, discovery, visualization, one-off inspection	Click drift, wrong project, undocumented changes.
Cloud Shell	Fast authenticated administration and labs	Persistent files or commands can still target the wrong project.
gcloud CLI	Repeatable operations and scripts	Implicit configuration, wrong account/project/region.
Client libraries/API	Applications, platforms, automation	Credential handling, retries, quotas, versioning.

4.2 gcloud Configuration

The gcloud CLI stores named configurations containing properties such as account, project, region, and zone. Named configurations reduce context mistakes when engineers operate multiple environments. Never assume the active context; verify it before a change.

```
gcloud auth list
gcloud config configurations list
gcloud config get-value project
gcloud config list
gcloud projects describe "$PROJECT_ID"
```

4.3 APIs and Service Enablement

Google Cloud services expose APIs that must often be enabled per project. Enabling an API can create service agents and affect quota or billing. Treat API enablement as infrastructure configuration: document it, automate it, and remove unused APIs only after dependency analysis.

```
gcloud services list --enabled
gcloud services enable compute.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
```

4.4 Authentication for Tools and Applications

Human CLI access commonly uses interactive user authentication. Applications should use attached service accounts or federation and Application Default Credentials rather than embedding secrets. Local development can use user ADC carefully, while production should use workload identity with narrowly scoped service accounts.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Command changes wrong project	Implicit active configuration	Run <code>gcloud config list</code> and use named configurations or explicit <code>--project</code> flags.
API call returns <code>PERMISSION_DENIED</code>	Wrong principal, role, resource, or disabled API	Check <code>gcloud auth list</code> , IAM policy, API enablement, organization policy, and audit logs.
Application works locally but not in cloud	Different ADC identity or service account	Inspect the runtime identity and grant the minimum required role to that identity.

Hands-on Lab 4: Create Safe gcloud Configurations

Prerequisites

- Cloud Shell or the Google Cloud CLI installed locally.

Implementation

1. Run `gcloud auth list` and identify the active principal.
2. Create a named configuration for the learning project.
3. Set the project, default region, and default zone.
4. Enable the Compute Engine API and list enabled services.
5. Run a read-only command with both implicit and explicit project selection.
6. Export command output as validation evidence without exposing tokens.

Validation Evidence

- The named configuration shows the correct account and project.
- The region and zone are explicit.
- Compute Engine API is enabled.
- Read-only output identifies the correct project number.

Cleanup

- Keep the configuration for later labs or delete it with `gcloud config configurations delete`.

Interview Questions and Model Answers

1. Why use named gcloud configurations?

To isolate account, project, region, and zone context and reduce cross-environment mistakes.

2. Is Cloud Shell automatically safe?

No. It is convenient and authenticated, but commands can still target the wrong project or make destructive changes.

3. What is Application Default Credentials?

A standard mechanism used by Google client libraries to locate credentials from the environment.

4. Why automate API enablement?

It is a project dependency that should be reviewable, repeatable, and consistent across environments.

Chapter Summary

- Use the console for discovery and CLI/API for repeatability.
- Always verify account, project, and location context.
- Treat enabled APIs as managed configuration.
- Use runtime identity instead of embedded credentials.

Chapter 5 - Cloud Identity and IAM Fundamentals

Controlling who can perform which actions on which Google Cloud resources

Learning Objectives

- Differentiate authentication, authorization, principals, roles, and policies.
- Use basic, predefined, and custom roles appropriately.
- Explain IAM inheritance and policy evaluation.
- Apply least privilege through groups and resource scope.

5.1 Principals and Authentication

A principal is an identity that can be granted access. Common principals include Google Accounts, groups, service accounts, workforce identities, and workload identities. Authentication proves identity; IAM authorization determines what the authenticated principal can do on a resource.

5.2 Roles and Permissions

Permissions are fine-grained actions such as reading an instance or creating a network. Roles bundle permissions. Basic roles such as Owner, Editor, and Viewer are broad and should be avoided for routine production access. Predefined roles are service-oriented and maintained by Google. Custom roles can narrow access but require lifecycle management.

Role Type	Use	Caution
Basic	Legacy/simple broad access	Too broad for most production duties.
Predefined	Common job or service access	May still include more than one task needs.
Custom	Precise enterprise duties	Must track permission changes and support status.

5.3 Policies, Inheritance, and Conditions

An IAM allow policy binds principals to roles on a resource. Access granted at an organization or folder can inherit to descendants. IAM Conditions can make bindings context-aware, for example time-bound or resource-name constrained, where supported. Deny policies and principal access boundary policies can impose additional restrictions in advanced governance designs.

- Grant groups rather than individual users for workforce access.
- Grant roles at the lowest sustainable scope.
- Use policy analysis and troubleshooting before adding more roles.
- Review unused permissions and stale memberships regularly.

5.4 Separation of Duties

Enterprise access separates identity administration, organization administration, network administration, security monitoring, billing, project ownership, and workload deployment. Privilege should be requestable, reviewable, time-bound where possible, and auditable. Emergency access must be protected and tested without becoming a daily shortcut.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
User receives access denied despite project role	Required permission is on another resource or blocked upstream	Identify the exact permission and resource; inspect inherited allow/deny and organization policy.
User has excessive access	Broad role inherited from folder or organization	Remove upstream basic role and replace it with group-based predefined/custom roles.
New custom role cannot include permission	Permission not supported in custom roles or wrong stage	Check permission support level and use an appropriate predefined role or redesign the duty.

Hands-on Lab 5: Build a Least-Privilege IAM Model

Prerequisites

- A learning project and two test identities or groups if available.

Implementation

1. List the current project IAM policy and identify broad roles.
2. Define Developer Viewer, Compute Operator, Network Administrator, Security Reviewer, and Billing Viewer duties.
3. Map each duty to predefined roles at the narrowest resource scope.
4. Grant one temporary test binding with an expiry condition if supported.
5. Use Policy Troubleshooter or a controlled test to validate allowed and denied actions.
6. Remove the temporary binding and capture audit evidence.

Validation Evidence

- The test identity can perform only the intended read or operation.
- An unauthorized action is denied.
- No Owner or Editor role is introduced.
- The change and removal are visible in audit evidence.

Cleanup

- Remove all test bindings and memberships.
- Retain the permission matrix and evidence.

Interview Questions and Model Answers

1. What is the difference between a permission and a role?

A permission represents an action; a role is a named collection of permissions that can be granted.

2. Why avoid basic roles?

Owner, Editor, and Viewer are broad and frequently violate least privilege and separation of duties.

3. How does IAM inheritance work?

Allow policies granted on higher hierarchy nodes can apply to descendant resources.

4. Why use groups?

Groups centralize workforce membership and make access review and offboarding more reliable.

Chapter Summary

- IAM answers who can do what on which resource.
- Predefined roles are generally preferable to basic roles.
- Scope and inheritance are as important as role selection.
- Access must be group-based, auditable, and periodically reviewed.

Chapter 6 - Service Accounts, Workload Identity Federation, and Privileged Access

Securing workload identities with short-lived credentials and controlled impersonation

Learning Objectives

- Explain service accounts as workload identities.
- Avoid service account keys through attached identities and federation.
- Use service account impersonation and short-lived tokens.
- Design privileged and emergency access with separation of duties.

6.1 Service Accounts

A service account is a Google Cloud identity intended for workloads and automation, not a shared human login. A service account can be attached to supported compute resources or impersonated by authorized principals. Its permissions come from IAM roles granted to the service account principal.

- Create a dedicated service account per application or trust boundary.
- Avoid default service accounts and broad automatic grants.
- Grant the service account only the resource roles it requires.
- Restrict who can act as or create tokens for the service account.

6.2 Why Keys Are Dangerous

A service account key is a long-lived credential that can be copied outside Google Cloud and is difficult to inventory once exported. Prefer attached service accounts for Google Cloud workloads, Workload Identity Federation for external workloads and CI/CD, and service account impersonation for administrators and automation.

6.3 Workload Identity Federation

Workload Identity Federation exchanges an identity from an external provider for short-lived Google Cloud credentials, eliminating the need to distribute service account keys. Providers can include external clouds, deployment pipelines, Kubernetes clusters, and standards-based identity systems. Trust configuration must restrict issuer, audience, subject, attributes, and allowed service accounts.

```

External identity provider
  ↓ signed assertion
Workload Identity Pool / Provider
  ↓ Security Token Service
Short-lived federated credential
  ↓ optional impersonation
Dedicated Google Cloud service account
  
```

6.4 Privileged and Break-Glass Access

Privileged access should be separate from daily access, time-limited where possible, protected by strong authentication, approved, logged, and reviewed. Service account impersonation lets a user obtain short-lived credentials without downloading a key. Emergency accounts require tested procedures, protected credentials, alerting, and post-use review.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Pipeline authentication fails	Issuer, audience, subject mapping, attribute condition, or IAM binding mismatch	Inspect provider configuration and token claims; validate principalSet and service-account token-creator bindings.
Workload can impersonate powerful service account	Overbroad Service Account User or Token Creator grant	Narrow the principal and service account; separate deployment and runtime identities.
Unknown key found in repository	Long-lived service account key leaked	Disable/delete key, rotate affected access, inspect audit logs, remove secret from history, implement federation.

Hands-on Lab 6: Replace a Static Key Design with Federation

Prerequisites

- A synthetic CI/CD scenario; actual external provider setup is optional.

Implementation

- Identify where a traditional pipeline would store a service account key.
- Define a dedicated deployment service account and its minimum roles.
- Design a workload identity pool and provider with issuer, audience, and repository/branch attributes.
- Define the principalSet binding allowed to impersonate the deployment service account.
- Create a threat model covering untrusted branches, forks, token replay, and privilege escalation.
- Write validation and revocation procedures.

Validation Evidence

- No long-lived key is required.
- Only the approved pipeline identity can impersonate the service account.
- Runtime and deployment service accounts are separate.
- Revocation can be performed without searching for distributed key files.

Cleanup

- If live resources were created, remove the provider, pool, and test IAM bindings.

Interview Questions and Model Answers

1. What is a service account?

A Google Cloud identity designed for workloads and automation.

2. Why prefer Workload Identity Federation?

It uses short-lived credentials and avoids distributing long-lived service account keys.

3. What is service account impersonation?

An authorized principal obtains temporary credentials to act as a service account without possessing its key.

4. What is a common privilege-escalation risk?

Granting broad ability to attach or impersonate a more privileged service account.

Chapter Summary

- Service accounts are workload identities, not shared users.
- Long-lived keys should be exceptional.
- Federation and impersonation provide short-lived access.
- Privileged access needs approval, logging, revocation, and review.

Chapter 7 - Resource Governance: Organization Policy, Tags, Labels, Quotas, and APIs

Applying preventive controls, metadata, service limits, and lifecycle standards at scale

Learning Objectives

- Differentiate organization policies, IAM, tags, labels, and quotas.
- Use organization policies to constrain risky configurations.
- Apply metadata for ownership, automation, and cost reporting.
- Treat APIs and quotas as production dependencies.

7.1 Organization Policy

IAM grants authorization to principals; Organization Policy constrains how resources may be configured. Constraints can restrict external IP use, allowed locations, domain sharing, service account key creation, and other behaviors. Policies inherit through the hierarchy and should be staged to avoid widespread disruption.

- Evaluate existing resources before enforcement.
- Use dry-run or simulation capabilities where available.
- Pilot in a lower folder or nonproduction project.
- Document exemptions, owners, expiry, and compensating controls.

7.2 Tags and Labels

Labels are key-value metadata used by many resources for organization, filtering, automation, and cost reporting. Resource Manager tags are hierarchical key-value metadata that can participate in policy and IAM conditions for supported services. Standardize names such as owner, environment, cost-center, data-classification, application, and expiry.

Metadata	Scope/Behavior	Common Use
Labels	Resource metadata; service support varies	Cost allocation, inventory, automation, filtering.
Resource Manager tags	Hierarchical tag keys/values with bindings	Policy conditions, governance, standardized enterprise classification.
Network tags	Compute networking selector concept	Firewall target selection; not a general governance label.

7.3 Quotas and Limits

Quotas protect shared capacity and control consumption. They can be global, regional, or metric-specific. Architecture and disaster recovery must include quota planning because a design can be correct yet undeployable. Monitor utilization and request increases before load tests or migrations.

7.4 APIs, Asset Inventory, and Lifecycle

Enabled APIs, service agents, projects, labels, policies, and quotas should be managed as code or through standardized project-factory workflows. Cloud Asset Inventory and Resource Manager provide visibility for governance and investigations. Project deletion and API disablement require dependency analysis and approved retention.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Resource creation blocked unexpectedly	Inherited organization policy or missing allowed value	Inspect effective policy at organization, folder, and project; validate constraint and exception.
Cost allocation incomplete	Missing or inconsistent labels	Enforce metadata in project factory/IaC and report unlabeled spend.
Deployment fails with quota exceeded	Regional or service quota exhausted	Inspect quota metric and region, reduce demand or request increase before deployment.

Hands-on Lab 7: Create a Governance Baseline

Prerequisites

- A learning project; organization-level permissions are optional.

Implementation

1. Define required labels and Resource Manager tags for owner, environment, cost center, and data class.
2. Inventory enabled APIs and document why each is required.
3. Review Compute Engine and project quotas for the selected region.
4. Design three organization policies: restrict key creation, restrict external IPs, and restrict resource locations.
5. Create a staged rollout and exception process.
6. Build a project-readiness checklist that includes IAM, logging, labels, budgets, APIs, and quotas.

Validation Evidence

- Every proposed control has an owner and rollback.
- Quotas are sufficient for the foundation labs.
- Metadata supports cost and ownership reporting.
- Policies are staged rather than immediately applied at the organization root.

Cleanup

- Remove any test labels, tags, or project policies created solely for the lab.

Interview Questions and Model Answers

1. How is Organization Policy different from IAM?

IAM controls who can act; Organization Policy constrains resource configuration.

2. What is the difference between labels and tags?

Labels are resource metadata; Resource Manager tags are hierarchical and can support policy or IAM conditions where integrated.

3. Why are quotas an architecture concern?

Capacity or recovery can fail if required quota is unavailable in the target scope or region.

4. Why manage API enablement?

Enabled services are dependencies that affect service agents, quota, security, and billing.

Chapter Summary

- Governance combines preventive policy, metadata, quotas, APIs, and lifecycle.
- Organization policies require staged rollout.
- Labels and tags need enterprise standards.
- Quota and API dependencies must be automated and monitored.

Chapter 8 - Compute Engine Foundations

Deploying secure, maintainable virtual machines with deliberate identity, storage, and lifecycle choices

Learning Objectives

- Explain Compute Engine instances, machine series, images, disks, and metadata.
- Create a VM through gcloud with secure defaults.
- Use service accounts, OS Login, Shielded VM, and startup scripts appropriately.
- Operate instance lifecycle, patching, snapshots, and cleanup.

8.1 Compute Engine Building Blocks

A Compute Engine VM combines a machine type, boot disk image, network interface, service account, metadata, scheduling behavior, and optional attached disks or accelerators. The VM is usually zonal, while supporting resources may be zonal, regional, or global.

Component	Decision	Operational Impact
Machine series/type	CPU, memory, architecture, workload optimization	Performance, quota, cost, live migration support.
Image/boot disk	OS family, hardened image, disk type and size	Patch baseline, startup time, storage cost, recovery.
Network interface	VPC, subnet, IP stack, external IP	Reachability, egress, firewall, exposure.
Service account	Runtime identity and API access	Blast radius and credential-free service access.

8.2 Secure VM Baseline

A secure baseline normally uses a dedicated service account, least privilege, no external IP unless required, OS Login or managed access, Shielded VM features, encrypted disks, centralized logging, patch management, vulnerability visibility, and controlled startup scripts. SSH keys in project metadata and broad default service-account roles create long-lived risk.

8.3 Images, Disks, and Metadata

Public images provide supported operating-system families. Custom images and image families enable controlled golden-image pipelines. Persistent Disk is durable block storage independent of the VM lifecycle when configured accordingly. Instance metadata supplies configuration but is not a secret store. Use Secret Manager and workload identity for sensitive values.

```
gcloud compute instances create gcp-foundation-vm \
  --zone=asia-south1-a \
  --machine-type=e2-micro \
  --subnet=foundation-subnet \
  --no-address \
  --service-account=vm-runtime@$PROJECT_ID.iam.gserviceaccount.com \
  --scopes=cloud-platform \
  --shielded-secure-boot \
  --labels=environment=lab,owner=student
```

8.4 Lifecycle and Operations

Production fleets should use templates and managed instance groups rather than hand-built VMs, which Part B covers. For individual instances, define patching, backup, monitoring, restart behavior, deletion protection where appropriate, disk retention, shutdown, and ownership. Stopped VMs can still incur disk and IP charges.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
SSH connection fails	No route, firewall, OS Login/IAM, or guest issue	Use IAP/serial diagnostics where approved; inspect firewall, IAM, OS Login, guest agent, and boot logs.
VM cannot call Google API	Wrong service account or missing IAM role	Inspect attached service account and runtime token; grant the minimum resource role.
VM deleted but cost remains	Persistent disk, snapshot, or static IP retained	List disks, snapshots, addresses, and billing SKUs; delete or intentionally retain.

Hands-on Lab 8: Deploy a Private Compute Engine VM

Prerequisites

- Learning project, Compute Engine API, VPC/subnet from Chapter 9 or create in sequence.

Implementation

- Create a dedicated VM runtime service account with no broad project role.
- Create a small Linux VM without an external IP and with Shielded VM features.
- Use a startup script to install a simple web service without embedding secrets.
- Connect through an approved private path or IAP design.
- Verify logs, machine identity, disk, network, and metadata.
- Stop, start, and delete the VM while checking retained disks and IPs.

Validation Evidence

- The VM has no public IPv4 address.
- The runtime identity is the dedicated service account.
- The service starts and logs are available.
- Cleanup leaves no orphaned disk or address.

Cleanup

- Delete the VM, boot disk, test firewall rule, and temporary service-account binding.

Interview Questions and Model Answers

1. What determines a VM identity?

The service account attached to the instance and the IAM roles granted to that service account.

2. Why avoid external IPs?

They increase exposure and can bypass private egress and inspection patterns.

3. Is metadata a secret store?

No. Use Secret Manager and identity-based access for secrets.

4. Why use managed instance groups later?

They provide template-based consistency, health repair, scaling, and multi-zone fleet operations.

Chapter Summary

- Compute Engine VMs combine compute, image, storage, network, and identity decisions.
- Use private networking and dedicated service accounts.
- Harden images and manage lifecycle explicitly.
- Cleanup must include disks, snapshots, and IP addresses.

Chapter 9 - VPC Networking Foundations

Designing global VPC networks with regional subnets, routes, firewall policy, DNS, and private service access

Learning Objectives

- Explain global VPC networks and regional subnets.
- Design IP ranges and custom-mode networks.
- Understand routes, firewall rules, service accounts, and network tags.
- Build private VM connectivity and controlled egress.

9.1 Global Network, Regional Subnets

A Google Cloud VPC network is a global resource. Its subnets are regional and can be used by resources in zones within that region. This differs from cloud designs where a virtual network itself is regional. Custom-mode networks are preferred for deliberate enterprise IP planning because subnets are created explicitly.

Object	Scope	Purpose
VPC network	Global	Connectivity, routes, firewall policy, peering, shared network boundary.
Subnet	Regional	Primary and secondary IP ranges for regional resources.
Route	Global with applicable destinations/tags	Determines packet next hop.
Firewall rule/policy	Global or policy-defined scope	Stateful distributed traffic filtering.

9.2 IP Planning and Shared VPC

Plan non-overlapping RFC1918 ranges with growth, GKE secondary ranges, hybrid networks, mergers, and regional expansion in mind. Shared VPC lets a host project provide centrally governed networks to service projects, separating network administration from workload ownership.

9.3 Firewall and Identity

Google Cloud VPC firewall rules are stateful and can target instances by service account or network tag. Service-account targeting provides an identity-oriented selector but requires careful IAM control over who can attach that service account. Deny and hierarchical firewall policies support centralized enforcement in advanced designs.

- Default deny ingress is a strong baseline.
- Allow only required sources, destinations, protocols, and ports.
- Do not expose SSH or RDP broadly; use approved administrative access.
- Enable firewall logging selectively for evidence and troubleshooting.

9.4 Private Access, DNS, and Egress

Private Google Access can let eligible resources without external IPs reach Google APIs and services. Cloud NAT provides outbound internet access for private instances without inbound exposure. Cloud DNS provides managed public and private DNS. Private Service Connect and service networking patterns provide private access to supported services, covered more deeply in Part B.

```

gcloud compute networks create foundation-vpc --subnet-mode=custom
gcloud compute networks subnets create foundation-subnet \
--network=foundation-vpc --region=asia-south1 --range=10.20.0.0/24 \
--enable-private-ip-google-access
gcloud compute firewall-rules create allow-health-from-admin \
--network=foundation-vpc --direction=INGRESS --action=ALLOW \
--rules=tcp:8080 --source-ranges=10.20.0.0/24

```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Instances in same VPC cannot connect	Firewall, wrong IP, service not listening, or route issue	Check effective firewall, flow logs, routes, guest listener, and subnet ranges.
Private VM cannot reach Google APIs	Private Google Access/DNS/route not configured	Enable subnet Private Google Access and validate DNS and allowed egress.
Peering or hybrid connection fails	Overlapping ranges or missing route exchange	Inspect IP plan, routes, dynamic route mode, and advertised prefixes.

Hands-on Lab 9: Build a Custom-Mode Private VPC

Prerequisites

- Learning project, Compute Engine API, and planned IP ranges.

Implementation

- Create a custom-mode VPC network.
- Create a regional subnet with Private Google Access.
- Create a dedicated service account and a narrowly targeted firewall rule.
- Deploy two private test VMs without external IPs.
- Test allowed application traffic and a denied port.
- Enable selected flow/firewall logging and capture evidence.

Validation Evidence

- The network is custom mode and the subnet is regional.
- VMs have only internal IP addresses.
- Only the approved port succeeds.
- A denied connection and an allowed connection are visible in evidence.

Cleanup

- Delete VMs, firewall rules, subnet, and VPC after the integrated capstone if not reused.

Interview Questions and Model Answers

1. Are Google Cloud VPC networks regional?

No. VPC networks are global; subnets are regional.

2. Why prefer custom-mode networks?

They give explicit control over subnet locations and address ranges.

3. What is Shared VPC?

A model where a host project centrally provides networks to workload service projects.

4. What does Cloud NAT provide?

Managed outbound translation for private resources without enabling unsolicited inbound internet connections.

Chapter Summary

- VPC networks are global and subnets are regional.
- IP planning must account for growth and hybrid connectivity.
- Firewall rules are stateful and can use identity-oriented targets.
- Private access and controlled egress reduce public exposure.

Chapter 10 - Cloud Billing, FinOps, and Foundation Capstone

Controlling cloud consumption through budgets, reporting, ownership, quotas, and automated cleanup

Learning Objectives

- Explain Cloud Billing accounts, project linkage, budgets, reports, and export.
- Distinguish budgets from hard spending controls.
- Use labels, ownership, and unit economics for FinOps.
- Integrate identity, hierarchy, networking, compute, governance, and cost controls in a capstone.

10.1 Cloud Billing Model

A Cloud Billing account pays for usage from linked projects. Billing access is separate from resource administration. Cost is influenced by resource configuration, runtime, location, data transfer, storage retention, committed use, licensing, support, and managed-service consumption. Engineers must understand the cost behavior of architecture decisions.

10.2 Budgets, Alerts, Reports, and Export

Budgets compare actual or forecast cost to thresholds and can notify recipients or publish programmatic notifications. They do not automatically stop all spending. Billing reports support interactive analysis, while billing export to BigQuery enables detailed allocation, dashboards, anomaly analysis, and unit economics.

Control	Purpose	Limitation
Budget alert	Notify at spend/forecast thresholds	Does not inherently prevent further spend.
Quota	Limit service consumption or allocation	Not a complete financial control and can disrupt workloads.
Labels/tags	Allocate cost and ownership	Requires consistent coverage and reporting design.
Automation	Stop/delete expired nonproduction resources	Must protect stateful and approved exceptions.

10.3 FinOps Operating Model

FinOps connects engineering, finance, and product teams. Effective practice assigns owners, exposes timely cost, defines allocation, measures business units such as cost per request or tenant, and creates an optimization backlog. Savings should not compromise reliability or create excessive operational toil.

- Use project and label design to make costs attributable.
- Review idle VMs, orphaned disks, static IPs, snapshots, logging volume, and data transfer.
- Use right-sizing evidence before commitments.
- Treat discounts as financial commitments with utilization risk.

10.4 Foundation Capstone Architecture

The capstone creates a secure, cost-governed application foundation: a dedicated project, explicit APIs, named gcloud configuration, group-based human access, dedicated VM service account, custom-mode VPC, regional subnet, private VM, restricted firewall, logging, labels, quota review, budget, and cleanup evidence.

```

Billing Account
|
Learning / Nonproduction Project
|— IAM groups and dedicated service account
|— Custom-mode global VPC
|   └─ asia-south1 private subnet
|— Private Compute Engine VM
|— Logging and audit evidence
|— Labels / ownership / expiry
|   └─ Budget alerts and cleanup automation

```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Budget exceeded without warning	Wrong scope, thresholds, recipients, or notification delay	Verify budget scope, thresholds, email/Pub/Sub recipients, and billing permissions.
Cost cannot be attributed	Shared project and missing metadata	Improve project boundaries, labels, billing export, and owner mapping.
Cleanup automation deletes required resource	Expiry logic lacks state/exception safeguards	Use allowlists, protection labels, approval, dry run, and recovery design.

Hands-on Lab 10: Build the Google Cloud Foundation Capstone

Prerequisites

- Completion of Chapters 1-9 and a dedicated learning project.

Implementation

- Create a named gcloud configuration and verify the active principal and project.
- Document hierarchy, billing account, enabled APIs, labels, owner, and expiry.
- Create group-based human access and a dedicated VM runtime service account.
- Build a custom-mode VPC, regional subnet, private access, and minimal firewall rules.
- Deploy a private VM with Shielded VM features and a simple startup workload.
- Create a budget, threshold notifications, and a cleanup checklist.
- Capture IAM, network, VM, audit, quota, and billing evidence.
- Perform cleanup and prove that no orphaned billable resources remain.

Validation Evidence

- The environment has no accidental broad basic-role grant.
- The VM has no external IP and uses a dedicated service account.
- Only the required network path is allowed.
- Budget and ownership controls are visible.
- Cleanup evidence shows no VM, orphaned disk, snapshot, or static IP.

Cleanup

- Delete all capstone compute and network resources unless retained intentionally.
- Optionally delete the learning project after exporting evidence and confirming no reuse.

Interview Questions and Model Answers

1. Do budgets cap Google Cloud spending automatically?

No. Budgets primarily monitor and notify; enforcement requires separate quotas, automation, policy, or operational action.

2. Why export billing data to BigQuery?

For detailed allocation, custom reporting, anomaly analysis, and unit-economics calculations.

3. What is unit economics?

Measuring cloud cost relative to a business output such as request, customer, order, or dataset processed.

4. What proves a foundation is production ready?

Validated identity, policy, networking, observability, quota, cost, recovery, ownership, and cleanup evidence.

Chapter Summary

- Billing is a separate administrative boundary linked to projects.
- Budgets alert but are not universal hard caps.
- FinOps needs ownership, allocation, measurement, and action.
- The capstone integrates secure identity, networking, compute, governance, and cost controls.

Appendix A - Google Cloud Command Reference

Purpose	Command
Verify identity	gcloud auth list
List configurations	gcloud config configurations list
Show active context	gcloud config list
Set project	gcloud config set project PROJECT_ID
List projects	gcloud projects list
Describe project	gcloud projects describe PROJECT_ID
List enabled APIs	gcloud services list --enabled
Enable Compute API	gcloud services enable compute.googleapis.com
List IAM policy	gcloud projects get-iam-policy PROJECT_ID
List service accounts	gcloud iam service-accounts list
List regions/zones	gcloud compute regions list; gcloud compute zones list
List VPC networks	gcloud compute networks list
List subnets	gcloud compute networks subnets list
List firewall rules	gcloud compute firewall-rules list
List VMs	gcloud compute instances list
List disks/addresses	gcloud compute disks list; gcloud compute addresses list
List quotas	gcloud compute project-info describe --project PROJECT_ID
Describe billing link	gcloud billing projects describe PROJECT_ID

Always review command help and current official documentation before destructive or production changes. Use explicit --project, --region, and --zone flags in automation where ambiguity would be dangerous.

Appendix B - Cross-Service Troubleshooting Matrix

Layer	First Evidence	Common Causes
Context	gcloud auth list; gcloud config list	Wrong account, project, region, zone, or configuration.
API	Enabled services and service usage errors	API disabled, service agent not ready, organization policy.
IAM	Policy Troubleshooter, effective policy, audit logs	Missing permission, wrong resource, inherited deny, impersonation failure.
Hierarchy	Resource parent and effective policy	Wrong folder/project placement, inherited controls, move impact.
Compute	Serial/boot logs, instance description, guest status	Image, startup script, disk, quota, service account, host/zone issue.

Layer	First Evidence	Common Causes
Network	Connectivity Tests, routes, firewall logs, flow logs	Firewall, route, DNS, overlapping ranges, missing NAT/private access.
Quota	Quota page/API and regional utilization	Insufficient regional capacity or service limit.
Billing	Reports, SKU, project linkage, labels	Orphan resources, data transfer, retention, missing allocation.

Appendix C - Foundation Production-Readiness Checklist

- ☐ Resource hierarchy and project ownership are documented.
- ☐ Production and nonproduction are separated.
- ☐ Billing account linkage, budgets, recipients, and export are defined.
- ☐ Groups are used for workforce access and broad basic roles are removed.
- ☐ Service accounts are dedicated per workload and long-lived keys are prohibited or formally excepted.
- ☐ gcloud configurations and automation use explicit context.
- ☐ Required APIs are managed and unused APIs are reviewed.
- ☐ Organization policies are staged, tested, and have governed exceptions.
- ☐ Labels/tags identify owner, environment, cost center, application, data class, and expiry.
- ☐ Regional and zonal failure domains are documented.
- ☐ Quota is sufficient in primary and recovery locations.
- ☐ VPC is custom mode with deliberate subnet ranges.
- ☐ Public IP use is justified and restricted.
- ☐ Firewall rules are least privilege and administrative access is controlled.
- ☐ VM images, patching, logging, service accounts, disks, and deletion behavior are defined.
- ☐ Audit logs and operational logs are retained and accessible.
- ☐ Cleanup, decommissioning, and orphan-resource detection are automated.
- ☐ Capstone validation includes allowed and denied tests plus cost evidence.

Appendix D - Glossary

Term	Definition
Application Default Credentials (ADC)	A standard credential-discovery mechanism used by Google client libraries.
Billing account	The account that pays for usage from linked Google Cloud projects.
Cloud Identity	Google identity platform for users and groups, often used to establish an organization resource.
Cloud Shell	Browser-based, authenticated shell with Google Cloud CLI and common tools installed.
Folder	Optional resource-hierarchy node used to group projects and inherit controls.
IAM	Identity and Access Management: principals, roles, permissions, and policies.

Term	Definition
Organization Policy	Constraints that limit resource configuration across the hierarchy.
Project	Core Google Cloud boundary for resources, APIs, quotas, IAM, billing association, and lifecycle.
Resource Manager tag	Hierarchical key/value metadata used for governance and supported policy conditions.
Service account	A workload identity used by applications, VMs, and automation.
Shared VPC	A model in which a host project supplies centrally governed networks to service projects.
VPC	A global virtual network containing regional subnets.
Workload Identity Federation	Federation that exchanges external identity assertions for short-lived Google Cloud credentials.
Zone	A deployment and failure domain within a Google Cloud region.

Official Google Cloud Documentation Index

Topic	Official URL
Google Cloud overview	https://docs.cloud.google.com/docs/overview
Regions and zones / Compute	https://docs.cloud.google.com/docs/compute-area
Resource hierarchy	https://docs.cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy
Projects	https://docs.cloud.google.com/resource-manager/docs/creating-managing-projects
IAM resource hierarchy access	https://docs.cloud.google.com/iam/docs/resource-hierarchy-access-control
Service account best practices	https://docs.cloud.google.com/iam/docs/best-practices-service-accounts
Workload Identity Federation	https://docs.cloud.google.com/iam/docs/workload-identity-federation
Google Cloud SDK and gcloud	https://docs.cloud.google.com/sdk
Cloud Shell	https://docs.cloud.google.com/shell/docs
Compute Engine instance creation	https://docs.cloud.google.com/compute/docs/instances/create-start-instance
VPC overview	https://docs.cloud.google.com/vpc/docs/overview
Cloud Billing documentation	https://docs.cloud.google.com/billing/docs
Budgets and alerts	https://docs.cloud.google.com/billing/docs/how-to/budgets
Cloud quotas	https://docs.cloud.google.com/docs/quotas/overview

Google Cloud services, names, regional availability, quotas, pricing, and certification requirements change. Validate current official documentation before production implementation or exam scheduling.

Completion Statement

Volume 8A is complete when the learner can create and explain a secure Google Cloud foundation, demonstrate least-privilege identity, use gcloud safely, deploy a private Compute Engine workload, troubleshoot VPC connectivity, prove budget and ownership controls, and clean up every billable resource with evidence.